React 19.3

Por qué tres setCount suman solo 1



Entiende el estado como una foto, la cola de actualizaciones y la inmutabilidad, y deja de perseguir contadores que no suman.

Chamba Lab · 2026 · Manual 12

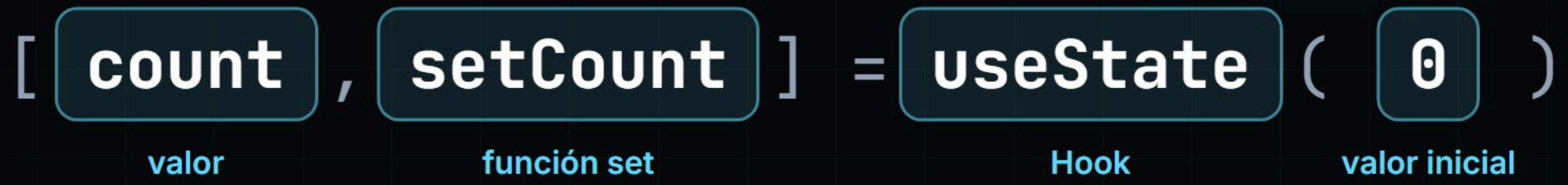


El estado es la memoria del componente

y cada render la ve como una foto fija

- React lo guarda fuera del componente, como en un estante
- Cuando lo cambias, React vuelve a llamar a tu componente
- Ese render recibe una foto nueva; la de antes no se toca

Anatomía: una línea, cuatro piezas



count

Los vasos del pedido en este render. Solo se lee.

setCount

Pide un render nuevo con otro valor.

useState

El Hook: React guarda el valor por ti.

0

Solo cuenta en el primer render.

Qué guardas: no todo valor se actualiza igual

Primitivos

Número, texto o booleano.

`count` `cliente` `enviado`

LOS MÁS COMUNES

Objetos

Datos que van juntos.

`pedido`

`{ ...pedido }`

COPIA AL CAMBIAR

Arrays

Listas que crecen o cambian.

`carrito` `map` `filter`

COPIA AL CAMBIAR

Derivado

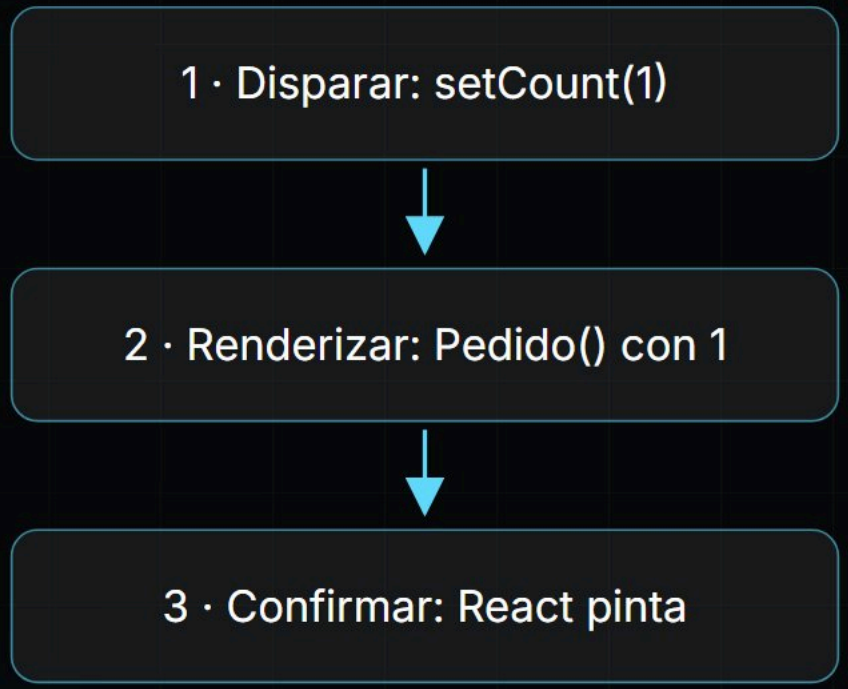
Se calcula al renderizar: no es estado.

`total`

`carrito.length`

NO LO GUARDES

Cómo funciona: disparar, renderizar, confirmar



Cada set pide un render nuevo; no cambia el actual

```
function Pedido() {
  const [count, setCount] = useState(0);
  // render 1: count = 0 · render 2: count = 1

  function sumarUno() {
    setCount(count + 1);
  }

  return (
    <button onClick={sumarUno}>
      Vasos: {count}
    </button>
  );
}
```



¿Qué muestra el alert?

count vale 0. ¿Sale 0 o 5? Decide antes de pasar.

```
function pedirCinco() {
  setCount(count + 5);
  alert(count);
}
```

Muestra 0: el handler ve la foto de su render

Pedido.jsx · render 1

```
// lo que escribes
function pedirCinco() {
  setCount(count + 5);
  alert(count);
}

// lo que corre en este render
function pedirCinco() {
  setCount(0 + 5); // pide un render con 5
  alert(0);
}
```

set no cambia `count`: pide otro render con el valor nuevo.

Cada render trae su foto y sus propios handlers.

Ni un `setTimeout` lo cambia: también verá 0.

El dolor: el botón «+3» suma 1



La QA pide 3 vasos y el pedido marca 1.
Las tres llamadas leen la misma foto: count vale 0 en todo el clic.

- Tres veces «reemplaza con 0 + 1»
- El último gana: 1

```
const [count, setCount] = useState(0);

function sumarTres() {
  setCount(count + 1); // 0 + 1
  setCount(count + 1); // 0 + 1
  setCount(count + 1); // 0 + 1
}

<button onClick={sumarTres}>+3</button>
// en pantalla: 1
```

La salida: pasa una función, no un valor



Cada función entra a la cola y espera su turno

- React las procesa en orden en el siguiente render
- Cada una recibe lo que devolvió la anterior

```
function sumarTres() {
  setCount(c ⇒ c + 1);
  setCount(c ⇒ c + 1);
  setCount(c ⇒ c + 1);
}
// cola: 0 → 1 → 2 → 3
```

c no es la foto: es lo que dejó la función anterior en la cola.

¿Y si mezclas? La cola se lee en orden

```
setCount(count + 5);
setCount(c ⇒ c + 1);
setCount(42);
```

- Un valor encola «reemplaza con…»
- Una función se encola y recibe el anterior

REEMPLAZA

`0 + 5`

recibe **0** · no lo usa

→ **5**

→

FUNCIÓN

`c ⇒ c + 1`

recibe **5**

→ **6**

→

REEMPLAZA

`42`

recibe **6** · no lo usa

→ **42**

Siguiente render: `count = 42`

Batching: un viaje con todo el pedido



- React espera a que termine el handler y hace un solo render
- Desde React 18 también agrupa en `setTimeout` y promesas
- Cada clic se procesa por separado
- `flushSync`: solo si necesitas el DOM ya

```
function confirmarPedido() {
  setCount(c ⇒ c + 1);
  setEstado('enviado');
  setNota('sin azúcar');
  // un solo render, al final
}
```

Objetos: no los cambies, reemplázalos

Mutar

```
pedido.cantidad = 3;
setPedido(pedido);
// mismo objeto: no hay render
```

Object.is ve el mismo objeto y lo ignora

Reemplazar

```
setPedido({
  ...pedido,
  cantidad: 3,
});
// objeto nuevo: React renderiza
```

Objeto nuevo: React ve el cambio

Anidados: copia cada nivel que tocas

Pedido.jsx

```jsx
const [pedido, setPedido] = useState({
  cliente: 'Rosa',
  cantidad: 1,
  entrega: { distrito: 'Surco' },
});

function cambiarDistrito(distrito) {
  setPedido({
    ...pedido,
    entrega: { ...pedido.entrega, distrito },
  });
}
```

El spread copia un solo nivel: `entrega` se copia aparte.

Copia desde el cambio hasta la raíz del objeto.

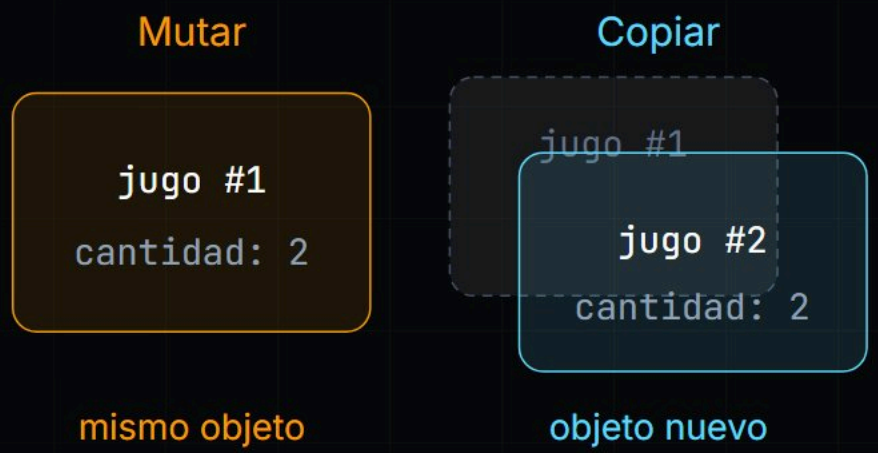
¿Muy anidado? Aplánalo o usa Immer (`useImmer`).

Arrays: el método decide si mutas

	Evita (muta)	Prefiere (crea uno nuevo)
Agregar	`push`, `unshift`	`[...carrito, jugo]`
Quitar	`pop`, `shift`, `splice`	`filter`, `slice`
Reemplazar	`splice`, `carrito[i] = …`	`map`
Ordenar	`sort`, `reverse`	Copia primero: `[...carrito]`

Si el método cambia el array original, úsalo solo sobre una copia.

El carrito: lista nueva, jugos copiados



- `[...carrito]` copia la lista, no los jugos
- Cambia un jugo con `{ ...j }`, nunca `j.cantidad++`

```
function sumarVaso(id) {
  setCarrito(carrito.map(j ⇒
    j.id === id
      ? { ...j, cantidad: j.cantidad + 1 }
      : j
  ));
}

function quitar(id) {
  setCarrito(carrito.filter(
    j ⇒ j.id !== id
  ));
}
```

Matriz de decisión: ¿valor o función?

	Valor `setCount(0)`	**Función** `setCount(c ⇒ c + 1)`
No depende del anterior	Lo natural	Funciona, pero sobra
Depende del anterior, un set por clic	Funciona: cada clic ve el valor al día	Funciona igual
Varios set en el mismo evento	Falla: todos leen la misma foto	Correcto: cada uno recibe el anterior

Si el nuevo valor sale del anterior, pasa una función. Si no, el valor. En objetos y arrays, siempre una copia nueva.

Lo que te llevas

- `count` es una foto: no cambia dentro de su render
- Si el valor sale del anterior: `setCount(c => c + 1)`
- Objetos y arrays: copia nueva, nunca mutar

> docs: react.dev/learn/state-as-a-snapshot

> docs: react.dev/learn/queueing-a-series-of-state-updates

> docs: react.dev/learn/updating-objects-in-state

> repo: github.com/Chamba-Lab